	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 6

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 9 de Octubre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **2**
2.1. Organización en Paquetes (Packages) 2
2.2. Compilación y Gestión del Classpath 2
2.3. Distribución mediante Archivos JAR 3
2.4. Buenas Prácticas y Generación de Javadoc 3

3. Desarrollo **3**

4. Resultados **5**
4.1. Buenas prácticas para archivos JAR 6
4.2. Buenas prácticas para JavaDoc 6
4.3. ¿Cuál es la mejor manera de generar un JavaDoc? 6
4.4. HTML 7

5. Conclusiones **8**

1. Introducción

- **Planteamiento del problema.** La acumulación de múltiples prácticas y proyectos en el desarrollo de aplicaciones en Java genera dispersión de código si este no se organiza mediante una estructura modular adecuada. El problema radica en la necesidad de agrupar, empaquetar, compilar y documentar de manera estandarizada las clases correspondientes a las prácticas del curso bajo un espacio de nombres jerárquico (`mx.unam.fi.die.poo.g7`), evitando conflictos de identificadores y facilitando su mantenimiento.

- **Motivación.** Es necesario resolver este problema porque el paradigma orientado a objetos exige modularidad, encapsulamiento y reutilización. La ausencia de paquetes formales dificulta la localización de clases, mientras que la falta de documentación técnica automatizada mediante Javadoc reduce la calidad del software al carecer de interfaces públicas claras y estandarizadas.

- **Objetivos.** Lo que se espera obtener de darle solución al problema es:

  - Consolidar un paquete general (`mx.unam.fi.die.poo.g7`) que estructure jerárquicamente las prácticas y proyectos del curso.
  - Aplicar las herramientas nativas del JDK (`javac`, `java`, `javadoc` y `jar`) para automatizar la compilación, ejecución y documentación web.
  - Analizar e implementar las mejores prácticas en el uso de archivos JAR y comentarios Javadoc normalizados con etiquetas de autoría y especificación.

2. Marco Teórico

2.1. Organización en Paquetes (Packages)

Un paquete es una agrupación de clases e interfaces relacionadas bajo un espacio de nombres común que permite estructurar bibliotecas, evitar colisiones de nombres mediante nombres cualificados (Fully Qualified Name) y controlar la accesibilidad de los miembros. Para incorporar una clase en un paquete, se declara la instrucción package al inicio del código fuente.[1]

2.2. Compilación y Gestión del Classpath

La compilación de código empaquetado se realiza con el comando javac utilizando la opción `-d`, la cual crea de forma automática la estructura de directorios destino requerida a partir de la ruta fuente. Durante la ejecución, el intérprete java requiere el uso del parámetro `-classpath` (o `-cp`) y el nombre totalmente calificado para localizar e interpretar correctamente los bytecodes.[1]

2.3. Distribución mediante Archivos JAR

Un archivo JAR (Java ARchive) es un formato comprimido basado en ZIP que agrupa múltiples clases y recursos en un solo archivo ejecutable o biblioteca sin necesidad de descompresión previa por parte de la JVM. Para que un JAR sea ejecutable, debe incluir un archivo de configuración META-INF/MANIFEST.MF que especifique la clase principal mediante la directiva Main-Class.[3]

2.4. Buenas Prácticas y Generación de Javadoc

La documentación técnica es fundamental para comunicar las intenciones del software en lenguaje natural. La forma más eficiente de generar documentación en formato HTML es ejecutando la herramienta javadoc desde la línea de comandos con un directorio de salida independiente mediante la opción `-d`:

```
javadoc -d doc -sourcepath src -subpackages mx.unam.fi.die.poo.g7
```

Entre las buenas prácticas destacan el uso exclusivo de bloques delimitados por `/** ... */`, la inclusión de descripciones concisas en la primera oración como resumen, y el empleo estricto de etiquetas estándar (`@author`, `@version`, `@param`, `@return`) en clases y métodos públicos.[2]

3. Desarrollo

Ejercicio 1

1. **Planteamiento de la consulta.** Consultar con una IA generativa las buenas prácticas para el uso de JAR y Javadoc, y cuál es la mejor manera de generar un Javadoc.

2. **Formulación de la pregunta.** Se redacta la consulta para que cubra los tres puntos solicitados:

   a) Buenas prácticas para el uso de JAR.

   b) Buenas prácticas para el uso de Javadoc.

   c) Mejor manera de generar un Javadoc.

3. **Consulta a la IA.** Se envía la pregunta a la IA generativa y se lee la respuesta completa.

4. **Análisis de la información sobre JAR.** Se revisan las recomendaciones que la IA da sobre la creación, organización y uso de archivos JAR.

5. **Identificación del método para generar el Javadoc.** Se identifica la forma de generar el Javadoc que la IA señala como la más adecuada y las razones que da.

6. **Verificación de la información.** Se contrasta lo obtenido con la documentación oficial de Oracle sobre `jar` y `javadoc`.

7. **Registro de la consulta.** Se anotan en el reporte la pregunta formulada, la IA utilizada y un resumen de la respuesta.

8. **Comparación entre integrantes.** Se reúnen las respuestas de los cinco integrantes y se contrastan para identificar coincidencias y diferencias.

9. **Conclusión.** Se resumen los puntos en común y se indica el método elegido para generar el Javadoc.

Ejercicio 2

1. **Inicio del trabajo.** Se abre la terminal de VS Code en la carpeta de usuario, donde se creará el paquete.

2. **Creación de las carpetas del paquete.** Se crean las carpetas anidadas `mx`, `unam`, `fi`, `die`, `poo` y `g7`, de modo que cada una quede dentro de la anterior y se forme la ruta `mx/unam/fi/die/poo/g7`.

3. **Creación de las subcarpetas de trabajo.** Dentro de `g7` se crean cuatro carpetas:

   a) `src`, para el código fuente (archivos `.java`).

   b) `lib`, para las librerías externas.

   c) `test`, para los archivos compilados (`.class`).

   d) `doc`, para los reportes de las prácticas.

4. **Creación de la ruta del paquete en `src`.** Dentro de `src` se crea nuevamente la ruta `mx/unam/fi/die/poo/g7`, porque Java requiere que las carpetas coincidan con el nombre del paquete. En ella se crea una carpeta por práctica (`practica01` a `practica05`).

5. **Incorporación del código.** Se guardan los archivos `.java` de cada práctica en la carpeta que le corresponde.

6. **Declaración del paquete.** Se agrega como primera línea de cada archivo el paquete al que pertenece, por ejemplo: package mx.unam.fi.die.poo.g7.practica01;

7. **Compilación.** Desde la carpeta `g7` se ejecuta: javac -d test ($findsrc - name$"$*.java$") La opción `-d` indica que los `.class` se guarden en `test`, y Java reproduce ahí la misma estructura de carpetas de `src`.

8. **Ejecución.** Se ejecuta cada programa indicando la carpeta de las clases compiladas y el nombre completo de la clase, por ejemplo: java -cp test mx.unam.fi.die.poo.g7.practica01

9. **Almacenamiento de los reportes.** Se guarda el reporte en PDF de cada práctica en la carpeta `doc/practicaXX`.

10. **Verificación final.** Se revisa que `src` contenga los `.java`, `test` los `.class`, `doc` los reportes y `lib` quede disponible para librerías externas.

11. **Cierre del trabajo.** Se confirma que la estructura quedó organizada y que todas las prácticas compilan sin errores.

```
mx/unam/fi/die/poo/g7/
├── src/
│   └── mx/unam/fi/die/poo/g7/
│       ├── practica01/
│       │   └── Calculadora.java
│       ├── practica02/
│       │   ├── Ejercicio2.java
│       │   └── TablaValores.java
│       ├── practica03/
│       │   └── Palindromo.java
│       ├── practica04/
│       │   ├── AnalizadorPalabras.java
│       │   └── Main.java
│       └── practica05/
│           ├── Empleado.java
│           ├── Fecha.java
│           ├── MainEmpleado.java
│           └── MainFecha.java
├── lib/
├── test/
│   └── mx/unam/fi/die/poo/g7/
│       ├── practica01/
│       ├── practica02/
│       ├── practica03/
│       ├── practica04/
│       └── practica05/
└── doc/
    ├── practica01/
    ├── practica02/
    ├── practica03/
    ├── practica04/
    └── practica05/
```

Figura 1: Ejercicio 2 diagrama

4. Resultados

Al preguntar a nuestra IA sobre buenas prácticas para uso de JAR y JavaDoc, obtuvimos lo siguiente:

4.1. Buenas prácticas para archivos JAR

- **Define el punto de entrada (Manifest):** Si el JAR es una aplicación ejecutable, asegúrate de incluir un archivo `MANIFEST.MF` que especifique la etiqueta `Main-Class`. Esto permite que el proyecto se pueda arrancar directamente desde la línea de comandos con `java -jar archivo.jar`.
- **Excluye el código fuente:** Un JAR debe contener exclusivamente los archivos binarios compilados (`.class`) y los recursos necesarios. Los archivos fuente (`.java`) deben mantenerse fuera del empaquetado final para mantener el archivo ligero.
- **Nomenclatura con versionado:** Nombra los archivos de forma semántica y añade la iteración del proyecto (por ejemplo, `algoritmos-ordenamiento-v1.0.jar`). En las entregas de proyectos de Programación Orientada a Objetos, esto ayuda a mantener un control de versiones limpio.
- **Modularidad (Fat JAR vs Thin JAR):** Para proyectos pequeños, está bien empaquetar todo en un solo archivo (Fat JAR). Para sistemas más grandes, es mejor mantener tus librerías externas en una carpeta separada y referenciarlas en el `Class-Path` del manifiesto.

4.2. Buenas prácticas para JavaDoc

- **Documenta el "quéz el "por qué", no el çómo":** Explica cuál es el propósito de una clase o qué devuelve un método. La lógica interna debe ir en comentarios de una línea (`//`) dentro del cuerpo del método, no en el JavaDoc.
- **La primera oración es crítica:** El motor de JavaDoc extrae la primera oración hasta el primer punto y la utiliza como resumen en las tablas HTML. Haz que esta primera línea sea directa y descriptiva.
- **Uso estricto de etiquetas:** Utiliza `@param` para describir cada argumento, `@return` para explicar el valor devuelto (omítelo si es `void`), y `@author` en la cabecera de las clases.
- **Enfoque en la API pública:** Prioriza siempre documentar las clases, interfaces y métodos `public` y `protected`. Los métodos `private` no requieren un bloque formal de JavaDoc.

4.3. ¿Cuál es la mejor manera de generar un JavaDoc?

Para desarrollo local usando Visual Studio Code y compilaciones por terminal, la herramienta nativa del JDK es la más directa. Abre tu terminal en la carpeta raíz de tu proyecto y ejecuta:

```
javadoc -d docs -sourcepath src -subpackages nombreDeTuPaquete
```

- `-d docs`: Crea una carpeta llamada `docs` donde depositará todos los archivos HTML generados.
- `-sourcepath src`: Especifica que tus archivos `.java` se encuentran dentro de la carpeta `src`.
- `-subpackages nombreDeTuPaquete`: Genera la documentación para todo el código dentro de ese paquete específico.

El estándar en la industria:
Cuando avances a proyectos más robustos, la mejor práctica es delegar la tarea a gestores de dependencias como Maven o Gradle. Con un solo comando como `mvn javadoc:javadoc`, estas herramientas construyen tu proyecto y generan la documentación HTML de forma automática.

4.4. HTML

Como resultado de la ejecución de la herramienta javadoc sobre el código fuente estructurado bajo la jerarquía de paquetes del proyecto, se generó de manera automatizada la documentación técnica en formato HTML dentro del directorio `doc`. A continuación se presentan las evidencias gráficas que validan la correcta organización y documentación de los componentes desarrollados a lo largo de las prácticas.

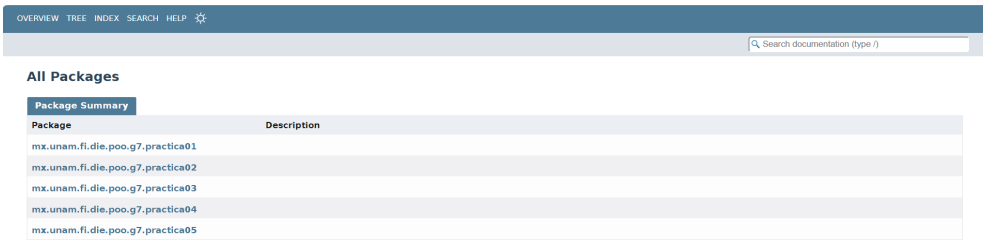


Figura 2: Vista general de los paquetes documentados bajo el espacio de nombres institucional.

La Figura 2 muestra el resumen de paquetes (Package Summary), donde se corrobora la correcta estructuración jerárquica de los subpaquetes correspondientes a cada práctica (`mx.unam.fi.die.poo.g7.practica01` a `practica05`), cumpliendo con los lineamientos de modularidad establecidos.

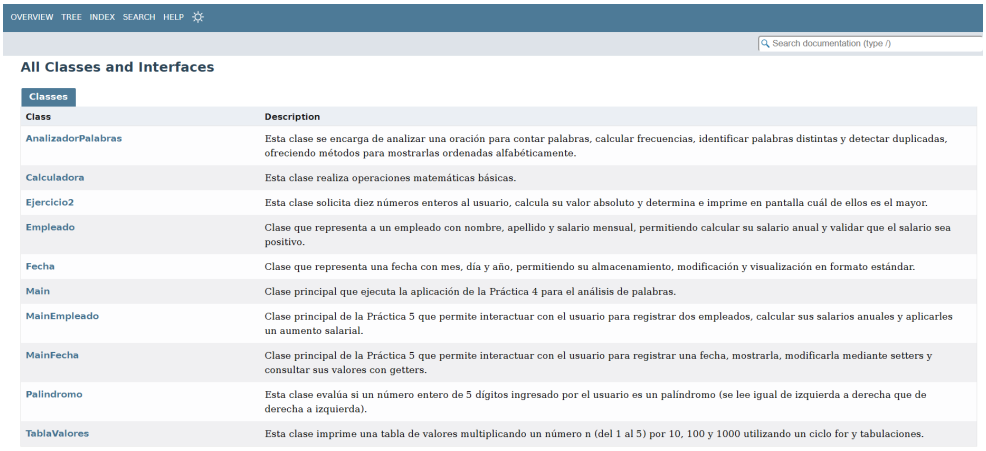


Figura 3: Índice general de clases e interfaces documentadas con Javadoc.

Asimismo, la Figura 3 ilustra el índice de clases e interfaces (All Classes and Interfaces), en el cual se enlistan los componentes desarrollados (tales como `AnalizadorPalabras`, `Calculadora`, `Empleado`, `Fecha`, entre otros). Cada componente muestra su respectiva descripción resumida derivada de los comentarios de documentación, verificando que la herramienta procesó adecuadamente las especificaciones públicas de las clases sin exponer detalles internos de implementación.

5. Conclusiones

La estructuración de proyectos en Java mediante paquetes formales y la separación de directorios de trabajo (`src`, `test`, `lib`, `doc`) es fundamental para mantener la modularidad y evitar colisiones de identificadores. De la aplicación de estos conceptos se concluyen tres principios técnicos esenciales:

- **Gestión automatizada:** El manejo adecuado del classpath y las herramientas del JDK estandarizan los procesos de compilación y ejecución.
- **Distribución eficiente:** El empaquetado en archivos JAR se confirma como el mecanismo óptimo para distribuir proyectos, dependiendo de la correcta configuración del MANIFEST.MF.
- **Documentación de calidad:** La generación de Javadoc enfocada en la API pública (public y protected) asegura la mantenibilidad del código al proveer interfaces claras sin exponer la lógica interna.

Referencias

[1] M. A. Nakayama C. y J. A. Solano Gálvez, *Manual de prácticas del Laboratorio de Modelos de Programación Orientada a Objetos (MADO-21)*, versión 02. Ciudad de México: Facultad de Ingeniería, UNAM, 2018.

[2] L. Joyanes Aguilar, *Fundamentos de programación. Algoritmos, estructuras de datos y objetos*, 4.ª ed. México: McGraw-Hill, 2008.

[3] J. Dean y R. Dean, *Introducción a la programación con Java*, 1.ª ed. México: McGraw-Hill, 2009.